UNIVERSIDADE PRESBITERIANA MACKENZIE
FACULDADE DE COMPUTAÇÃO E INFORMÁTICA

Filipe Soares – RA 10734981
Miguel Soares – RA 10734935
Julio Freire – RA 10734961

PROJETO EXTENSIONISTA: SISTEMA INTEGRADO PARA A INDÚSTRIA FARMACÊUTICA

Documento 2 – Arquitetura de Software
Estrutura, componentes, tecnologias e funcionamento (padrão MVC)

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO....3

2 ARQUITETURA DE SOFTWARE....3

3 PADRÃO ARQUITETURAL ADOTADO: MVC....3

3.1 Conceito e responsabilidades no padrão MVC....3

3.2 Fluxo de execução....4

4 JUSTIFICATIVA DA ESCOLHA ARQUITETURAL....4

4.1 Alternativas avaliadas e descartadas....5

4.2 Comparativo entre as arquiteturas....5

5 ESTRUTURA DO SISTEMA E COMPONENTES....6

5.1 Visão geral....6

5.2 Camadas e componentes....6

5.3 Decisões estruturais....6

6 TECNOLOGIAS UTILIZADAS....7

7 FUNCIONAMENTO DO SISTEMA....8

7.1 Fluxo de execução no código....8

7.2 Código-fonte da camada Model....8

7.3 Código-fonte das camadas View, Controller e testes....10

7.4 Cenários de teste e saída da execução....13

7.5 Síntese dos resultados....14

REFERÊNCIAS....15

1 INTRODUÇÃO

Este documento apresenta a arquitetura de software do Sistema Integrado para a Indústria Farmacêutica (RastroFarma), dando continuidade à especificação de requisitos (Documento 1) e à modelagem elaborada na Unified Modeling Language — UML (Documento 3). O objetivo é descrever a definição arquitetural adotada, o padrão Model-View-Controller (MVC), e a implementação prática de um recorte funcional em linguagem Python.

Para atender ao escopo proposto, selecionou-se o fluxo de controle de qualidade sobre lotes de matéria-prima recebidos em quarentena, correspondente ao requisito funcional RF-QUA-01 e ao caso de uso UC03 (Avaliar lote de material). Conforme as convenções estabelecidas no projeto, todos os artefatos de código e diagramação preservam rigorosamente os mesmos identificadores de classes, atributos, métodos e estados em letras maiúsculas, tais como QUARENTENA, APROVADO e REPROVADO, em alinhamento às diretrizes de Boas Práticas de Fabricação (BRASIL, 2022) e de proteção de dados (BRASIL, 2018).

2 ARQUITETURA DE SOFTWARE

A arquitetura de software compreende o conjunto de decisões estruturais relativas aos componentes de um sistema, às formas de comunicação entre eles e aos princípios que orientam sua evolução. Tais decisões determinam atributos essenciais de qualidade, incluindo manutenibilidade, escalabilidade, desempenho, testabilidade e resiliência. Por se tratarem de definições de difícil reversão, a escolha arquitetural deve considerar o domínio do problema, o tamanho da equipe e o momento evolutivo do produto.

3 PADRÃO ARQUITETURAL ADOTADO: MVC

3.1 Conceito e responsabilidades no padrão MVC

Proposto por Trygve Reenskaug no final da década de 1970 no ambiente Smalltalk, o padrão MVC organiza internamente a aplicação por meio da separação de responsabilidades em três camadas distintas:

a) **Model (Modelo):** concentra os dados, o estado das entidades e as regras de negócio da aplicação, sem possuir qualquer conhecimento sobre como essas informações serão exibidas ou formatadas;

b) **View (Visão):** encarrega-se exclusivamente da apresentação dos dados ao usuário, podendo ser materializada por telas, páginas HTML ou respostas estruturadas em texto e JSON;

c) **Controller (Controlador):** recebe a entrada ou requisição externa, aciona o Model para processar as regras de negócio ou consultar a base de dados e seleciona a View responsável por renderizar a resposta.

3.2 Fluxo de execução

O funcionamento típico da arquitetura MVC desenvolve-se em cinco etapas sequenciais: (1) o usuário envia uma requisição ou comando; (2) o Controller recebe a solicitação e invoca o Model; (3) o Model recupera ou atualiza os dados aplicando as validações de negócio e devolve o resultado ao Controller; (4) o Controller entrega os dados processados à View; e (5) a View formata e devolve a resposta final ao usuário.

O Quadro 1 sintetiza as características arquiteturais do padrão MVC que fundamentam sua adoção para o recorte do Sistema Integrado para a Indústria Farmacêutica.

Quadro 1 – Síntese das características do padrão arquitetural MVC

Critério arquitetural	Caracterização no padrão MVC
Foco principal	Separar apresentação (View), dados e regras de negócio (Model) e controle de fluxo (Controller).
Nível de aplicação	Organização interna da aplicação, ideal para sistemas administrativos e módulos de informação de complexidade baixa a média.
Acoplamento e complexidade	Acoplamento moderado e complexidade baixa, permitindo reutilizar a mesma regra de negócio do Model em diferentes formatos de saída na View.
Vantagens e limitações	Oferece clara divisão de responsabilidades e evolução paralela, exigindo cuidado para evitar controladores sobrecarregados (*fat controllers*) em sistemas extensos.

Fonte: Elaborado pelos autores com base no material de Arquitetura de Sistemas.

4 JUSTIFICATIVA DA ESCOLHA ARQUITETURAL

A decisão arquitetural seguiu a regra de ouro apresentada no material da disciplina: a melhor arquitetura é a mais simples que atenda às necessidades reais, preparada para evoluir. Com base nesse critério, o padrão MVC foi escolhido pelos seguintes motivos:

a) **complexidade baixa e foco interno:** organiza a aplicação internamente sem exigir infraestrutura pesada;

b) **separação clara de responsabilidades:** isola regras de negócio (Model), controle de fluxo (Controller) e exibição (View);

c) **adequação a sistemas de informação:** é indicado para módulos administrativos, cadastros e validações de complexidade baixa a média;

d) **mesma regra em várias saídas:** o Model não sabe como será exibido, funcionando tanto com telas quanto com saída JSON no console.

4.1 Alternativas avaliadas e descartadas

As demais arquiteturas estudadas foram descartadas para este recorte, conforme o Quadro 2.

Quadro 2 – Arquiteturas descartadas para o recorte funcional

Arquitetura	Complexidade	Motivo do descarte
SOA (Orientada a Serviços)	Alta	Focada em integrar múltiplos sistemas legados via barramentos (ESB) e contratos pesados.
Orientada a Eventos	Média-alta	Exige brokers (Kafka/RabbitMQ), mensageria assíncrona e consistência eventual.
Hexagonal	Média	Exige múltiplas portas, interfaces e adaptadores — exagero para um recorte direto.
Microsserviços	Alta	Exige deploy distribuído, bancos separados por serviço e infraestrutura DevOps pesada.

Fonte: Elaborado pelos autores.

4.2 Comparativo entre as arquiteturas

O Quadro 3 reproduz o comparativo resumido das arquiteturas apresentado no material de Arquitetura de Sistemas, que serviu de base para a decisão.

Quadro 3 – Comparativo resumido das arquiteturas de software

Critério	MVC	SOA	Eventos	Hexagonal	Microsserv.
Foco	Separar apresentação, dados e controle	Integrar sistemas corporativos	Comunicação assíncrona	Isolar o domínio	Deploy e escala independentes
Nível	Interno da aplicação	Entre sistemas	Estilo de comunicação	Interno da aplicação	Distribuição do sistema
Acoplamento	Moderado	Moderado (ESB)	Muito baixo	Baixo (portas)	Baixo (APIs)
Complexidade	Baixa	Alta	Média-alta	Média	Alta
Exemplo	Sistema escolar	Banco multicanal	Detecção de fraude	Motor de frete	Netflix

Fonte: Material de Arquitetura de Sistemas (Arquiteturas de Software: MVC, SOA, Orientada a Eventos, Hexagonal e Microsserviços).

5 ESTRUTURA DO SISTEMA E COMPONENTES

5.1 Visão geral

O sistema integrado tem a função de concentrar as informações de cada lote em um único lugar, integrando as áreas de recebimento de materiais, controle de qualidade, produção, liberação do lote, estoque e logística e recolhimento (recall). A arquitetura modular permite adoção gradual por laboratórios menores, ampliando o acesso à tecnologia.

O recorte implementado nesta etapa corresponde ao controle de qualidade de lotes de matéria-prima e articula as classes da vista de materiais, qualidade e estoque com a classe de controle de acesso. Sob a arquitetura MVC, essas classes são distribuídas nas três camadas estruturais, garantindo que as regras de estado permaneçam encapsuladas no Model.

5.2 Camadas e componentes

O Quadro 4 apresenta os componentes do recorte funcional em cada camada e o papel de cada um no padrão MVC.

Quadro 4 – Componentes do recorte funcional por camada MVC

Camada	Componentes	Responsabilidade
Model (dados e regras)	LoteMaterial, DecisaoQualidade, Usuario, MateriaPrima, Fornecedor e as enumerações StatusLoteMaterial, Decisao e PerfilUsuario	Concentra os dados, o estado das entidades e as regras de negócio: valida o perfil QUALIDADE, verifica se o status é QUARENTENA, valida a justificativa e a validade e grava a DecisaoQualidade.
Controller (controle)	AvaliacaoQualidadeController	Recebe a entrada do usuário, busca o LoteMaterial, aciona as regras do Model e repassa o resultado à View.
View (apresentação)	AvaliacaoQualidadeView	Formata os dados processados em JSON estruturado no console, sem lógica de negócio; não altera estados nem acessa o banco de dados.
Armazenamento (simulado)	Dicionário banco_lotes em memória	Simula o banco de dados, associando o identificador de cada lote ao respectivo objeto LoteMaterial.

Fonte: Elaborado pelos autores.

5.3 Decisões estruturais

A distribuição das classes nas camadas MVC segue quatro decisões estruturais diretamente alinhadas à especificação original:

a) **encapsulamento no Model:** a classe LoteMaterial concentra o atributo status (do tipo enumerado StatusLoteMaterial) e os métodos de negócio

aprovar(), reprovar(), estaVencido(), podeSerConsumido() e baixarSaldo(). O lote nasce em QUARENTENA e somente transita para APROVADO ou REPROVADO mediante validação de estado e de permissão;

b) **rastreabilidade de decisões e perfis no Model:** cada avaliação gera uma instância de DecisaoQualidade vinculada ao LoteMaterial e ao Usuario que a registrou, armazenando a enumeração Decisao (APROVADO ou REPROVADO), a justificativa e o carimbo de data e hora;

c) **isolamento da apresentação na View:** a classe AvaliacaoQualidadeView não altera estados nem acessa o banco de dados, limitando-se a receber os objetos de domínio e formatar respostas estruturadas em JSON para exibição no console;

d) **orquestração no Controller:** a classe AvaliacaoQualidadeController expõe os métodos acionados pelo usuário (avaliarLoteMaterial e consultarHistoricoLote), recupera as entidades de domínio, invoca os métodos do Model e repassa o resultado para a View.

6 TECNOLOGIAS UTILIZADAS

A codificação do recorte funcional foi desenvolvida no arquivo `projeto_mvc.py`, utilizando orientação a objetos direta em Python e apenas os módulos padrão json e datetime, sem dependência de interface gráfica. A camada View renderiza as saídas em formato JSON estruturado, demonstrando o desacoplamento entre as regras de negócio do domínio farmacêutico e a apresentação dos resultados.

Quadro 5 – Tecnologias e recursos utilizados no recorte

Tecnologia / recurso	Uso no projeto
Python (orientação a objetos)	Linguagem de implementação das classes das camadas Model, View e Controller, no arquivo projeto_mvc.py.
Módulo padrão json	Serialização das respostas da View em JSON estruturado (json.dumps).
Módulo padrão datetime	Datas de recebimento e validade dos lotes (date) e carimbo de data e hora das decisões (datetime).
Dicionário em memória	Simulação do banco de dados de lotes (banco_lotes).
Console (terminal)	Interface de execução e exibição dos resultados, sem interface gráfica.
UML / PlantUML	Especificação formal do Diagrama de Classes do recorte (ver Documento 3).

Fonte: Elaborado pelos autores.

7 FUNCIONAMENTO DO SISTEMA

7.1 Fluxo de execução no código

O Quadro 6 mostra como os cinco passos do padrão MVC acontecem dentro do script Python.

Quadro 6 – Fluxo de execução MVC no arquivo projeto_mvc.py

Passo	Etapa	O que acontece no código
1	Requisição	O script chama controller.avaliarLote() passando ID, usuário, decisão e justificativa.
2	Controller	Busca o lote no dicionário em memória e chama lote.aprovar() ou lote.reprovar().
3	Model	Valida o perfil QUALIDADE, checa se status == QUARENTENA e grava DecisaoQualidade.
4	Retorno	O Controller recebe a decisão criada pelo Model e a envia para a View.
5	View (JSON)	AvaliacaoQualidadeView formata os dados em JSON legível e imprime no console.

Fonte: Elaborado pelos autores.

7.2 Código-fonte da camada Model

O trecho a seguir implementa as classes de domínio da camada Model (Fornecedor, MateriaPrima, Usuario, DecisaoQualidade e LoteMaterial), preservando os atributos e métodos definidos no Diagrama de Classes.

Listagem 1 – Camada Model (projeto_mvc.py)

```
import json
from datetime import date, datetime

# ==========================================
# MODEL - Classes de Dominio e Regras
# ==========================================

class Fornecedor:
    def __init__(self, id, razao_social, cnpj, qualificado=True):
        self.id = id
        self.razao_social = razao_social
        self.cnpj = cnpj
        self.qualificado = qualificado


class MateriaPrima:
    def __init__(self, id, codigo, nome, unidade_medida, estoque_minimo):
        self.id = id
        self.codigo = codigo
        self.nome = nome
        self.unidade_medida = unidade_medida
        self.estoque_minimo = estoque_minimo
```

```
class Usuario:
    def __init__(self, id, nome, login, senha, perfil, ativo=True):
        self.id = id
        self.nome = nome
        self.login = login
        self.senhaHash = "hash_" + senha
        self.perfil = perfil
        self.ativo = ativo

    def autenticar(self, senha):
        return self.ativo and self.senhaHash == ("hash_" + senha)

    def temPermissao(self, acao):
        if not self.ativo:
            return False
        if acao == "AVALIAR_LOTE_MATERIAL" and self.perfil == "QUALIDADE":
            return True
        return False


class DecisaoQualidade:
    def __init__(self, id, decisao, justificativa, usuario):
        self.id = id
        self.decisao = decisao
        self.justificativa = justificativa
        self.usuario = usuario
        self.dataHora = datetime.now().strftime("%Y-%m-%d %H:%M:%S")


class LoteMaterial:
    def __init__(self, id, numero_lote, lote_fornecedor, materia_prima, fornecedor,
qtd_recebida, data_recebimento, data_validade):
        self.id = id
        self.numeroLote = numero_lote
        self.loteFornecedor = lote_fornecedor
        self.materiaPrima = materia_prima
        self.fornecedor = fornecedor
        self.quantidadeRecebida = qtd_recebida
        self.saldoAtual = qtd_recebida
        self.dataRecebimento = data_recebimento
        self.dataValidade = data_validade
        self.status = "QUARENTENA"
        self.historicoDecisoes = []

    def estaVencido(self):
        return self.dataValidade < date.today()

    def podeSerConsumido(self, qtd):
        return self.status == "APROVADO" and not self.estaVencido() and self.saldoAtual >=
qtd and qtd > 0

    def baixarSaldo(self, qtd):
        if not self.podeSerConsumido(qtd):
            raise Exception("Lote nao pode ser consumido ou saldo insuficiente.")
        self.saldoAtual -= qtd
        if self.saldoAtual == 0:
```

```
            self.status = "ESGOTADO"

    def aprovar(self, usuario, justificativa):
        if not usuario.temPermissao("AVALIAR_LOTE_MATERIAL"):
            raise Exception(f"Acesso negado: usuario '{usuario.nome}' ({usuario.perfil}) nao
tem perfil QUALIDADE.")
        if self.status != "QUARENTENA":
            raise Exception(f"Bloqueado: o lote '{self.numeroLote}' esta com status
'{self.status}'. So lotes em QUARENTENA podem ser avaliados.")
        if not justificativa or justificativa.strip() == "":
            raise Exception("A justificativa e obrigatoria.")
        if self.estaVencido():
            self.status = "VENCIDO"
            raise Exception("Bloqueado: o lote esta vencido.")

        self.status = "APROVADO"
        nova_decisao = DecisaoQualidade(len(self.historicoDecisoes) + 1, "APROVADO",
justificativa, usuario)
        self.historicoDecisoes.append(nova_decisao)
        return nova_decisao

    def reprovar(self, usuario, justificativa):
        if not usuario.temPermissao("AVALIAR_LOTE_MATERIAL"):
            raise Exception(f"Acesso negado: usuario '{usuario.nome}' ({usuario.perfil}) nao
tem perfil QUALIDADE.")
        if self.status != "QUARENTENA":
            raise Exception(f"Bloqueado: o lote '{self.numeroLote}' esta com status
'{self.status}'. So lotes em QUARENTENA podem ser avaliados.")
        if not justificativa or justificativa.strip() == "":
            raise Exception("A justificativa e obrigatoria para reprovar o lote.")

        self.status = "REPROVADO"
        nova_decisao = DecisaoQualidade(len(self.historicoDecisoes) + 1, "REPROVADO",
justificativa, usuario)
        self.historicoDecisoes.append(nova_decisao)
        return nova_decisao
```

Fonte: Elaborado pelos autores.

7.3 Código-fonte das camadas View, Controller e testes

O bloco complementar apresenta a implementação da AvaliacaoQualidadeView, do AvaliacaoQualidadeController e a rotina de execução dos cinco cenários de teste.

Listagem 2 – Camadas View, Controller e testes (projeto_mvc.py)

```
# ========================================
# VIEW - Exibicao dos Resultados (JSON/Texto)
# ========================================

class AvaliacaoQualidadeView:
    def renderizarSucesso(self, lote, decisao):
        dados = {
            "status_requisicao": "SUCESSO",
            "lote": lote.numeroLote,
```

```
            "materia_prima": lote.materiaPrima.nome,
            "novo_status": lote.status,
            "decisao": {
                "id": decisao.id,
                "resultado": decisao.decisao,
                "justificativa": decisao.justificativa,
                "analista": decisao.usuario.nome,
                "data_hora": decisao.dataHora
            }
        }
        return json.dumps(dados, indent=4, ensure_ascii=False)

    def renderizarHistorico(self, lote):
        lista = []
        for d in lote.historicoDecisoes:
            lista.append({
                "id": d.id,
                "decisao": d.decisao,
                "justificativa": d.justificativa,
                "usuario": d.usuario.nome,
                "data_hora": d.dataHora
            })
        dados = {
            "lote": lote.numeroLote,
            "status_atual": lote.status,
            "historico": lista
        }
        return json.dumps(dados, indent=4, ensure_ascii=False)

    def renderizarErro(self, mensagem):
        dados = {
            "status_requisicao": "ERRO",
            "motivo": mensagem
        }
        return json.dumps(dados, indent=4, ensure_ascii=False)


# ========================================
# CONTROLLER - Intermediario entre View e Model
# ========================================

class AvaliacaoQualidadeController:
    def __init__(self, banco_lotes, view):
        self.banco_lotes = banco_lotes
        self.view = view

    def avaliarLote(self, lote_id, usuario, decisao, justificativa):
        lote = self.banco_lotes.get(lote_id)
        if not lote:
            return self.view.renderizarErro("Lote nao encontrado.")

        try:
            if decisao == "APROVADO":
                reg = lote.aprovar(usuario, justificativa)
            elif decisao == "REPROVADO":
                reg = lote.reprovar(usuario, justificativa)
            else:
```

```
                return self.view.renderizarErro("Decisao invalida.")

            return self.view.renderizarSucesso(lote, reg)
        except Exception as erro:
            return self.view.renderizarErro(str(erro))

    def consultarHistoricoLote(self, lote_id):
        lote = self.banco_lotes.get(lote_id)
        if not lote:
            return self.view.renderizarErro("Lote nao encontrado.")
        return self.view.renderizarHistorico(lote)


# ==========================================
# TESTES (Simulando o uso do sistema)
# ==========================================

if __name__ == "__main__":
    # Criando dados basicos para testar
    fornecedor1 = Fornecedor(1, "BioQuimica Insumos Ltda", "12.345.678/0001-99")
    mp1 = MateriaPrima(10, "MP-001", "Paracetamol Po", "kg", 50.0)

    lote101 = LoteMaterial(101, "LM-2026-001", "FORN-889", mp1, fornecedor1, 200.0,
date(2026, 11, 1), date(2028, 11, 1))
    lote102 = LoteMaterial(102, "LM-2026-002", "FORN-890", mp1, fornecedor1, 150.0,
date(2026, 11, 2), date(2028, 11, 2))

    analista = Usuario(1, "Dra. Helena Costa", "helena.qualidade", "123456", "QUALIDADE")
    almoxarife = Usuario(2, "Carlos Mendes", "carlos.almox", "654321", "ALMOXARIFE")

    # Dicionario simulando o banco de dados em memoria
    banco_lotes = {
        101: lote101,
        102: lote102
    }

    view = AvaliacaoQualidadeView()
    controller = AvaliacaoQualidadeController(banco_lotes, view)

    print("--- CENARIO 1: Aprovando lote 101 em Quarentena (UC03) ---")
    print(controller.avaliarLote(101, analista, "APROVADO", "Laudo conferido e material
aprovado."))

    print("\n--- CENARIO 2: Reprovando lote 102 em Quarentena (UC03 / UC14) ---")
    print(controller.avaliarLote(102, analista, "REPROVADO", "Embalagem violada e com
umidade."))

    print("\n--- CENARIO 3: Tentativa de aprovacao por Almoxarife (Bloqueio de Perfil) ---")
    print(controller.avaliarLote(101, almoxarife, "APROVADO", "Tentando aprovar pelo
almoxarifado."))

    print("\n--- CENARIO 4: Tentativa de avaliar lote que nao esta mais em Quarentena ---")
    print(controller.avaliarLote(101, analista, "REPROVADO", "Tentando mudar lote ja
aprovado."))

    print("\n--- CENARIO 5: Consultando historico do lote 101 ---")
    print(controller.consultarHistoricoLote(101))
```

Fonte: Elaborado pelos autores.

7.4 Cenários de teste e saída da execução

A rotina de testes executa cinco cenários:

a) cenário 1: aprovação do lote 101 em quarentena (UC03);

b) cenário 2: reprovação do lote 102 em quarentena (UC03 / UC14);

c) cenário 3: tentativa de aprovação por almoxarife (bloqueio de perfil – RNF-03);

d) cenário 4: tentativa de avaliar lote que não está mais em quarentena;

e) cenário 5: consulta ao histórico do lote 101.

A execução do arquivo projeto_mvc.py gera a saída a seguir, comprovando o funcionamento das regras de negócio e dos bloqueios de segurança.

Listagem 3 – Saída da execução no console

```
--- CENARIO 1: Aprovando lote 101 em Quarentena (UC03) ---
{
    "status_requisicao": "SUCESSO",
    "lote": "LM-2026-001",
    "materia_prima": "Paracetamol Po",
    "novo_status": "APROVADO",
    "decisao": {
        "id": 1,
        "resultado": "APROVADO",
        "justificativa": "Laudo conferido e material aprovado.",
        "analista": "Dra. Helena Costa",
        "data_hora": "2026-11-08 12:50:52"
    }
}

--- CENARIO 2: Reprovando lote 102 em Quarentena (UC03 / UC14) ---
{
    "status_requisicao": "SUCESSO",
    "lote": "LM-2026-002",
    "materia_prima": "Paracetamol Po",
    "novo_status": "REPROVADO",
    "decisao": {
        "id": 1,
        "resultado": "REPROVADO",
        "justificativa": "Embalagem violada e com umidade.",
        "analista": "Dra. Helena Costa",
        "data_hora": "2026-11-08 12:50:52"
    }
}

--- CENARIO 3: Tentativa de aprovacao por Almoxarife (Bloqueio de Perfil) ---
{
    "status_requisicao": "ERRO",
    "motivo": "Acesso negado: usuario 'Carlos Mendes' (ALMOXARIFE) nao tem perfil
QUALIDADE."
}
```

```
--- CENARIO 4: Tentativa de avaliar lote que nao esta mais em Quarentena ---
{
    "status_requisicao": "ERRO",
    "motivo": "Bloqueado: o lote 'LM-2026-001' esta com status 'APROVADO'. So lotes em
QUARENTENA podem ser avaliados."
}

--- CENARIO 5: Consultando historico do lote 101 ---
{
    "lote": "LM-2026-001",
    "status_atual": "APROVADO",
    "historico": [
        {
            "id": 1,
            "decisao": "APROVADO",
            "justificativa": "Laudo conferido e material aprovado.",
            "usuario": "Dra. Helena Costa",
            "data_hora": "2026-11-08 12:50:52"
        }
    ]
}
```

Fonte: Elaborado pelos autores.

7.5 Síntese dos resultados

A execução dos cenários demonstra que:

- as regras de negócio estão protegidas dentro do Model;
- o fluxo é controlado pelo Controller e formatado pela View;
- a arquitetura é simples, direta e aderente ao diagrama UML.

REFERÊNCIAS

ARQUITETURAS de software: MVC, SOA, orientada a eventos, hexagonal e microsserviços. Material didático (slides) da disciplina. [S. l.: s. n.], [2026].

BRASIL. Agência Nacional de Vigilância Sanitária. **Resolução da Diretoria Colegiada RDC nº 658, de 30 de março de 2022**. Dispõe sobre as Diretrizes Gerais de Boas Práticas de Fabricação de Medicamentos. Brasília, DF: ANVISA, 2022.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

OBJECT MANAGEMENT GROUP. **Unified Modeling Language (UML)**: specification, version 2.5.1. Needham: OMG, 2017.